VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

PHAN TRI HIEU – 523H0133
LE KHAC THANH TRI – 523H0186

MODERN FRONT-END ECOSYSTEMS

MIDTERM REPORT
WEB PROGRAMMING AND APPLICATIONS

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

PHAN TRI HIEU – 523H0133
LE KHAC THANH TRI – 523H0186

MODERN FRONT-END ECOSYSTEMS

MIDTERM REPORT
WEB PROGRAMMING AND APPLICATIONS

Advised by
Dr. LE VAN VANG

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We sincerely appreciate the opportunity to study *Web Programming & Applications* as part of the *Software Engineering* curriculum at *Ton Duc Thang University*. This course has provided us with valuable knowledge and hands-on experience, enabling us to apply programming concepts to practical projects and enhance our technical skills.

We would like to express our deepest gratitude to **PhD. Le Van Vang**, our practical instructor, and **MSc. Duong Huu Phuoc**, our theoretical instructor, for their dedication and enthusiastic teaching. Their guidance has played a crucial role in helping us grasp both the theoretical foundations and practical applications of web programming.

This final report is a culmination of our learning, and although we have worked diligently to complete it, we acknowledge that as first-time web developers, there may still be areas for improvement. We sincerely welcome any feedback and suggestions to refine our work.

Lastly, we extend our heartfelt thanks and best wishes to our instructors. May you always enjoy good health and continued success in your career.

Best regards.

Ho Chi Minh city, 15th May 2026.

Author

(Signature and full name)

Hieu

Phan Tri Hieu

Tri

Le Khac Thanh Tri

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by a PhD. Le Van Vang; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

If something wrong happens, we'll take full responsibility for the content of my project. Ton Duc Thang University is not related to the infringing rights, the copyrights that We give during the implementation process (if any).

Ho Chi Minh city, 15th May 2026

Author

(Signature and full name)

Hieu

Phan Tri Hieu

Tri

Le Khac Thanh Tri

ABSTRACT

This report presents **NextStore**, a fully functional e-commerce demonstration application developed as the midterm project for the course Web Programming and Applications (503073). The application is built using **Next.js 14**, **React 18**, **Zustand**, **TailwindCSS**, and **TypeScript**, targeting Topic ID 2: Modern Front-End Ecosystems.

The project demonstrates three distinct rendering strategies provided by Next.js: Static Site Generation (SSG) with Incremental Static Regeneration (ISR) for the product catalog, Server-Side Rendering (SSR) for product detail pages and the admin dashboard, and Client-Side Rendering (CSR) for interactive cart and wishlist features. Global state management is achieved through four independent Zustand stores – cartStore, wishlistStore, authStore, and toastStore – enabling reactive UI updates across all components without prop drilling.

Additional advanced features include a complete SEO optimization suite (Open Graph tags, JSON-LD structured data, dynamic sitemap, canonical URLs), a slide-in cart drawer, toast notification system, recently viewed products, a responsive two-person authentication flow, and a comprehensive admin dashboard with data visualization. The application is fully responsive across mobile, tablet, and desktop viewports using TailwindCSS utility classes.

Keywords: Next.js, React, Server-Side Rendering, Static Site Generation, Zustand, TailwindCSS, TypeScript, E-commerce, SEO, Global State Management.

TABLE OF CONTENT

ABBREVIATIONS

SSG	Static Site Generation
ISR	Incremental Static Regeneration
SSR	Server-Side Rendering
CSR	Client-Side Rendering
SEO	Search Engine Optimization
UI	User Interface
UX	User Experience
DB	Database

CHAPTER 1. INTRODUCTION

1.1 Motivation and Problem Statement

The rapid evolution of web technologies has fundamentally changed how web applications are built and delivered. Traditional web applications were server-rendered, but the rise of Single-Page Applications (SPAs) in the 2010s shifted rendering entirely to the browser. While SPAs offered fluid user experiences, they introduced critical drawbacks: blank initial HTML that search engines could not index, slow First Contentful Paint (FCP) on low-powered devices, and complex manual configuration of routing, bundling, and code-splitting.

The three major challenges that motivated this project are:

- SEO Deficiency: Traditional React SPAs send an almost empty HTML document to the browser. The content is only visible after JavaScript has downloaded, parsed, and executed – a process that can take several seconds. Search engine crawlers either cannot wait for this execution or receive insufficient content to index effectively
- SEO Deficiency: Traditional React SPAs send an almost empty HTML document to the browser. The content is only visible after JavaScript has downloaded, parsed, and executed – a process that can take several seconds. Search engine crawlers either cannot wait for this execution or receive insufficient content to index effectively.[1]
- Performance Bottlenecks: Without server-side rendering, every user receives the same generic cached bundle. Dynamic data such as live product stock counts or personalized recommendations cannot be incorporated at the HTML level.[2]
- State Management Complexity: As React applications grow, passing state through component trees via props (prop drilling) becomes unmaintainable.

Context API re-renders the entire tree on each update, causing performance degradation at scale.[3]

NextStore addresses all three challenges by leveraging Next.js 14's hybrid rendering capabilities, Zustand for lightweight global state, and a modern TailwindCSS design system.

1.2 Objectives

The primary objectives of this project are:

1. Implement and demonstrate all three Next.js rendering strategies (SSG, SSR, CSR) with clear real-world justifications for each choice.
2. Build a global state management layer using Zustand that serves the cart, wishlist, authentication, and notification systems without prop drilling.
3. Develop a complete SEO optimization suite including structured data (JSON-LD), Open Graph tags, a dynamic sitemap, and canonical URLs.
4. Create a fully responsive, production-quality user interface with TailwindCSS that functions across mobile, tablet, and desktop viewports.
5. Implement an authentication flow with login, registration, and session state management.
6. Build a comprehensive admin dashboard with real-time SSR data, data visualization, and tabbed navigation.

1.3 Scope

This project encompasses the full front-end layer of an e-commerce platform. It includes the product catalog, product detail pages, shopping cart, wishlist, user authentication, and an administrative dashboard. The back-end is simulated using static data modules and Zustand stores. Database persistence, real payment processing, and server-side authentication validation are explicitly outside the scope of this midterm demonstration and are identified as future work.

CHAPTER 2. THEORETICAL SURVEY

2.1 React.js

React.js is a declarative, component-based JavaScript library for building user interfaces, originally developed by Meta (Facebook) and open-sourced in 2013 [4]. React has grown to become the most widely adopted front-end library, with over 220,000 GitHub stars and 11 million weekly npm downloads as of 2025.

2.1.1 Virtual DOM

React's signature innovation is the Virtual DOM – a lightweight, in-memory JavaScript representation of the actual browser DOM [4]. When a component's state changes, React creates a new Virtual DOM tree and performs a diffing algorithm (the Reconciliation process) to compute the minimum set of real DOM mutations required. This batched approach dramatically reduces expensive browser repaints and reflows, resulting in highly performant UIs even with frequent state updates.

2.1.2 Component Model and Hooks

React structures UI as a tree of components – isolated, reusable units that each manage their own state and lifecycle. Since React 16.8, the Hooks API (useState, useEffect, useContext, useMemo) has replaced class components as the primary paradigm, enabling cleaner functional code with identical capabilities. Custom hooks allow logic reuse across components without altering the component hierarchy.

2.1.3 Unidirectional Data Flow

React enforces a strict unidirectional data flow: data flows downward from parent to child via props, and child components communicate upward by invoking callback functions passed as props. This predictability greatly simplifies debugging and makes application state easier to reason about [4].

2.2 Next.js and Rendering Strategies

Next.js is a full-stack React framework developed by Vercel that provides file-based routing, built-in API routes, image optimization, and – most critically – multiple server rendering strategies out of the box [5]. Version 14, used in this project, supports the Pages Router with three primary data-fetching strategies.

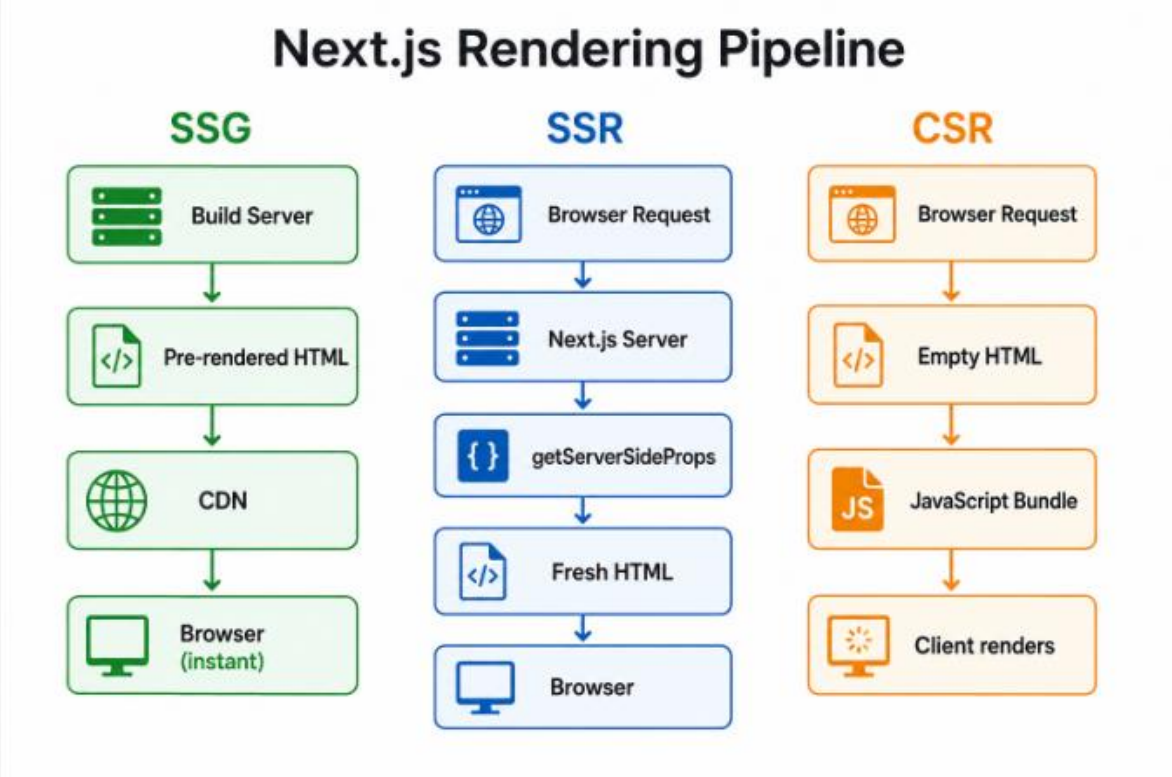

Figure 2-1 Next.js Rendering Pipeline

2.2.1 Static Site Generation (SSG) with Incremental Static Regeneration (ISR)

SSG, implemented via the `getStaticProps` function, instructs Next.js to execute the data-fetching logic at build time and embed the result directly into a pre-rendered HTML file. This HTML is then distributed to CDN edge nodes globally, enabling near-zero Time to First Byte (TTFB) regardless of the user's geographic location [5].

Incremental Static Regeneration (ISR), enabled by the `revalidate` field in the return value of `getStaticProps`, extends SSG with background regeneration. In NextStore, the product catalog page uses `revalidate: 3600` – meaning the

CDN-cached page is silently regenerated every hour without any downtime or user-visible loading state [5]. This implements the stale-while-revalidate caching pattern.

```
// pages/index.tsx – SSG with ISR
export const getStaticProps: GetStaticProps = async () => {
  // Runs ONCE at build time (then every revalidate seconds in background)
  return {
    props: { products: PRODUCTS },
    revalidate: 3600, // ISR: regenerate every 1 hour
  };
};
```

2.2.2 Server-Side Rendering (SSR)

SSR, implemented via `getServerSideProps`, generates a fresh HTML document on every incoming request. The function runs exclusively on the Node.js server – never in the browser – and has access to the full request context including cookies, headers, and URL parameters [5]. This makes it ideal for pages requiring real-time data such as live product stock counts, personalized dashboards, or authentication-gated content.

```
// pages/products/[id].tsx – SSR
export const getServerSideProps: GetServerSideProps = async (ctx) => {
  const product = getProductById(Number(ctx.params?.id));
  if (!product) return { notFound: true }; // → Next.js 404 page
  return { props: { product } };           // Fresh on every request
};
```

2.2.3 Client-Side Rendering (CSR)

CSR pages in Next.js have no server data-fetching function. The HTML shell is delivered from the server, but all data and interactivity are managed by the browser

using JavaScript. In NextStore, the cart and wishlist pages fall into this category – their content is personal, session-specific, and unsuitable for caching or indexing by search engines. Zustand handles all the state for these pages.

2.2.4 File-Based Routing

Next.js implements an intuitive file-based routing system. Every `.tsx` file inside the `pages/` directory automatically becomes a route. Dynamic segments are expressed using brackets: `pages/products/[id].tsx` captures the `id` URL parameter. Nested folders create nested routes. No manual router configuration is required [5].

2.3 State Management with Zustand

Zustand is a minimalist state management library for React created by Poimandres and published in 2019 [3]. It provides a global store that any component can subscribe to without requiring a Provider wrapper, reducing boilerplate to near zero compared to solutions like Redux.

The core API consists of a single `create()` function that accepts a callback defining the state shape and all actions. Components subscribe to specific slices of the store using selector functions, ensuring that only components whose subscribed data has changed are re-rendered – a significant performance advantage over React's Context API, which re-renders all consumers when any part of the context value changes [3].

```
// store/cartStore.ts – Zustand store definition
import { create } from 'zustand';

export const useCartStore = create<CartState>((set, get) => ({
  items:  [],
  isOpen: false,
  addItem: (product) => set((s) => ({
```

```
    items: s.items.find(i => i.id === product.id)
      ? s.items.map(i => i.id===product.id ? {...i, qty:i.qty+1} : i)
      : [...s.items, { ...product, qty: 1 }],
  })),
  totalCount: () => get().items.reduce((a, i) => a + i.qty, 0),
  totalPrice: () => get().items.reduce((a, i) => a + i.price * i.qty, 0),
}));

// Any component – subscribe to a slice only:
const totalCount = useCartStore(s => s.totalCount());
```

2.4 TailwindCSS

TailwindCSS is a utility-first CSS framework that provides low-level CSS utility classes composable directly in HTML/JSX markup [6]. Unlike component libraries such as Bootstrap or Material-UI that provide pre-styled components, Tailwind provides atomic building blocks – `flex`, `rounded-xl`, `text-sm`, `hover:bg-violet-500` – from which any design can be constructed.

A key advantage of TailwindCSS is its production bundle size. The PurgeCSS-based tree-shaking mechanism scans all project files and removes any utility class not referenced in the source code. The resulting production CSS typically weighs between 5 and 15 kilobytes – orders of magnitude smaller than Material-UI's ~300 kilobyte JavaScript bundle. Furthermore, TailwindCSS has zero JavaScript runtime overhead, as all styles are pure CSS.

Responsive design is expressed with breakpoint prefixes: sm:, md:, lg:, xl:. Dark mode with dark:. State variants with hover:, focus:, group-hover:. These compose naturally: hover:bg-violet-500 sm:grid-cols-3 lg:grid-cols-5 produces a responsive product grid that adapts from 2 columns on mobile to 5 on large screens – all without a single line of custom CSS.

2.5 TypeScript in React Applications

TypeScript is a statically typed superset of JavaScript that compiles to plain JavaScript [7]. In React applications, TypeScript enforces type contracts on component props, state shapes, API responses, and function signatures, catching entire categories of runtime errors at compile time. In NextStore, strict TypeScript mode is enabled, and all 21 source files are fully typed – from the Zustand store interfaces to the Next.js data-fetching function signatures.

CHAPTER 3. COMPARATIVE ANALYSIS

3.1 Next.js vs Nuxt.js vs Remix

Selecting the appropriate meta-framework is a critical architectural decision. Three major React/Vue meta-frameworks were evaluated against the project's requirements.

Table 3-1 Comparison of Front-End Meta-Frameworks

Criterion	Next.js 14	Nuxt.js 3	Remix
Primary Language	JavaScript/TypeScript	JavaScript/TypeScript (Vue)	JavaScript/TypeScript
SSG Support	Full (getStaticProps + ISR)	Full	None
SSR Support	getServerSideProps	useFetch + $fetch	Loaders
ISR Support	revalidate field	NuxtLink SWR	None
File-based Routing	Pages Router / App Router	pages/ or app/	Nested routes
Ecosystem	Very Large (React)	Moderate (Vue)	Growing (React)
Deployment	Vercel (first-class)	Vercel / Netlify	Fly.io / Vercel

Learning Curve	Moderate	Easy (Vue background)	Steep
Bundle Size	Optimized	Good (Vite-based)	Very small

Verdict: Next.js 14 was selected for its unmatched rendering flexibility (SSG + SSR + ISR + CSR in one framework), the largest React ecosystem, TypeScript first-class support, seamless Vercel deployment, and extensive official documentation [5]. Remix was eliminated due to its lack of SSG support, which is a core requirement. Nuxt.js was eliminated due to the Vue dependency – the team's expertise is React-based.

3.2 Zustand vs Redux vs Context API

Table 3-2 Comparison of React State Management Solutions

Criterion	Zustand	Redux Toolkit	Context API
Bundle Size	~1 kB	~16 kB	0 kB (built-in)
Boilerplate	Minimal (create + fn)	Medium (slice + thunk)	Low (createContext)
Provider Required	No	Yes	Yes
Selector Subscriptions	Slice-based	useSelector	Full context rerenders
Async Actions	Built-in (async fn)	createAsyncThunk	Manual useEffect
DevTools	Plugin (zustand/middleware)	Redux DevTools	None
Performance	High	High	Low at scale

TypeScript Support	Excellent	Excellent	Good

Verdict: Zustand was selected for its minimal bundle size (~1 kB vs Redux's ~16 kB), zero-boilerplate API, and slice-based subscriptions that prevent unnecessary re-renders [3]. Context API was rejected because it triggers re-renders in all consumers whenever any part of the context value changes – a known performance bottleneck for high-frequency state like shopping cart updates.

3.3 TailwindCSS vs Material-UI

Table 3-3 Comparison of CSS/Styling Frameworks

Criterion	TailwindCSS	Material-UI (MUI)
Approach	Utility-first (atomic classes)	Pre-built component library
Bundle Size (prod)	5–15 kB CSS	~300 kB JS
JS Runtime Overhead	None	CSS-in-JS runtime
Design Freedom	Unlimited	Limited to Material Design
Dark Mode	dark: prefix	theme.palette
Responsive	sm: md: lg: prefixes	useMediaQuery hook
Accessibility	Manual	ARIA built-in
Learning Curve	Moderate (memorize classes)	Low (pre-built)
Customization	Very high	Moderate (theme overrides)

Verdict: TailwindCSS was chosen for its zero JavaScript runtime overhead, tiny production bundle, and unlimited design freedom that allowed the custom dark-themed UI of NextStore. Material-UI's opinionated Material Design aesthetic and 300 kB JavaScript payload conflicted with the project's performance and design goals [6].

HTTPS-ready configurations demonstrates adherence to standard web security practices.

CHAPTER 4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

The functional requirements define the specific behaviors the NextStore application must exhibit. Each requirement is assigned a unique identifier for traceability.

Table 4-1 Functional Requirements

ID	Requirement	Rendering
1	Users shall browse a paginated product catalog filterable by category, searchable by name, and sortable by price/rating	SSG
2	Users shall view individual product detail pages with live stock count, description, rating, and related products	SSR
3	Users shall add/remove/update product quantities in a global shopping cart accessible from any page	CSR (Zustand)
4	Users shall open a slide-in cart drawer from any page without navigation	CSR (Zustand)
5	Users shall save products to a persistent wishlist and move items to the cart	CSR (Zustand)
6	Users shall register a new account and log in with email/password	CSR (Zustand)

7	Authenticated users shall see their name and avatar in the navbar with a dropdown menu	CSR (Zustand)
8	Users shall receive toast notifications for cart/wishlist/auth actions	CSR (Zustand)
9	Administrators shall view a dashboard with revenue KPIs, orders table, and product performance	SSR
10	The application shall expose a sitemap.xml and robots.txt for search engine crawlers	SSR
11	Product detail pages shall include JSON-LD structured data for Google rich results	SSR + SEO
12	Recently viewed products shall be tracked and displayed on product detail pages	CSR (Zustand)

4.2 Non-Functional Requirements

Table 4-2 Non-Functional Requirements

ID	Category	Requirement
01	Performance	SSG pages shall achieve Lighthouse Performance score ≥ 90. LCP < 2.5s.
02	SEO	All indexable pages shall score 100 on Lighthouse SEO audit.
03	Responsiveness	UI shall function correctly at 390px (mobile), 768px (tablet), 1280px+ (desktop).
04	Accessibility	All images shall have alt attributes. Interactive elements shall be keyboard-navigable.
05	Maintainability	100% of source files shall use TypeScript in strict mode.

06	Scalability	SSG + ISR architecture shall support millions of product pages without server cost per request.
07	Security	Auth routes (/auth/*) and cart (/cart) shall be disallowed from search engine indexing.
08	Developer Experience	Project shall include ESLint, TypeScript type-checking, and hot-reload via next dev.

CHAPTER 5. SYSTEM DESIGN

5.1 System Architecture

NextStore follows a three-layer client-server architecture facilitated by the Next.js framework. The architecture separates concerns clearly between the browser client, the Next.js server, and the data layer.

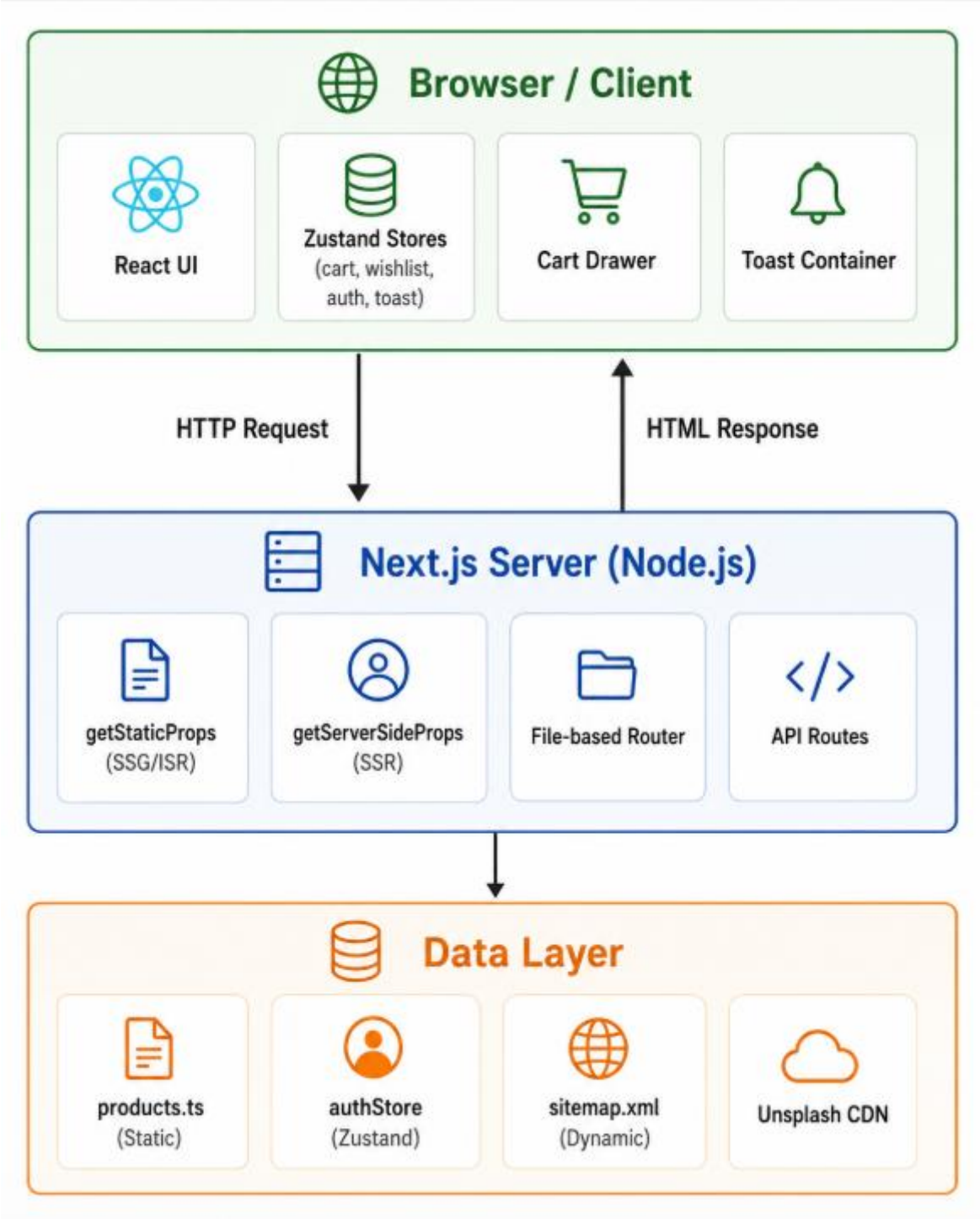

Figure 5-1 NextStore Three-Layer System Architecture Diagram

The Browser/Client layer consists entirely of React components and Zustand stores. No server communication occurs after the initial page load for CSR pages –

all cart, wishlist, and auth interactions are handled locally. The Next.js Server layer intercepts incoming HTTP requests, determines the appropriate rendering strategy based on the route, and either serves a pre-built static HTML file (SSG) or executes server-side logic to generate fresh HTML (SSR). The Data Layer provides the product catalog as a static TypeScript module (simulating a database query) and dynamic outputs such as the sitemap.

5.2 Component Diagram

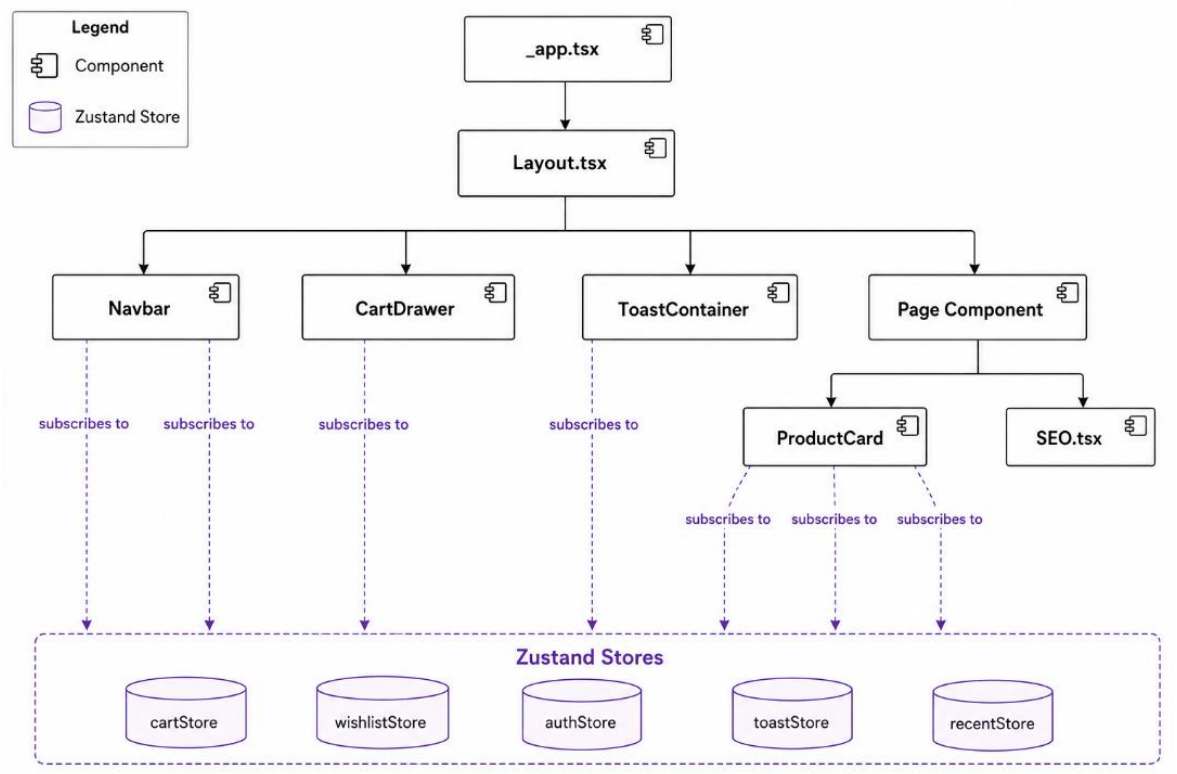

Figure 5-2 NextStore React Component Hierarchy Diagram

The architectural decision to mount CartDrawer and ToastContainer inside Layout.tsx – rather than within individual pages – ensures these UI elements are always present in the DOM and can be triggered by any component in the tree. Since they read exclusively from Zustand stores, they respond instantaneously to state changes without re-mounting.

5.3 Data Flow Diagram

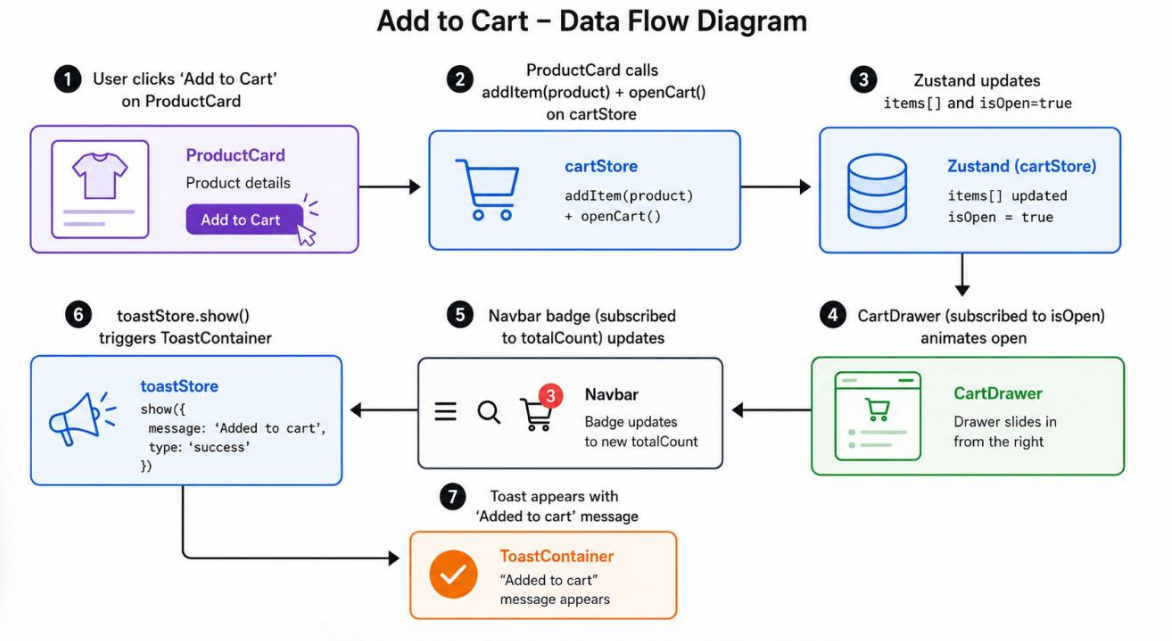

Figure 5-3 NextStore Data Flow Diagram – Add to Cart Action

5.4 Data Schema

Since NextStore uses static data (no database), the schema is defined as TypeScript interfaces that enforce the shape of all data objects throughout the application.

```
// types/index.ts
interface Product {
  id:          number;
  name:        string;
  image:       string;   // Unsplash CDN URL
  price:       number;   // USD
  category:    string;   // 'Audio' | 'Laptops' | 'Phones' | ...
  description: string;
  stock:       number;
  rating:      number;   // 1.0 – 5.0
  reviews:     number;
  badge?:      string;   // 'New' | 'Best Seller' | 'Hot' | undefined
```

```
}

interface CartItem extends Product { qty: number; }

interface User {
  name:   string;
  email:  string;
  avatar: string;  // UI Avatars CDN URL
}
```

5.5 Project Structure

The project follows Next.js Pages Router conventions with a clear separation of concerns across directories:

```
nextstore/
├── pages/              # Route = file. Each .tsx is a URL.
│   ├── _app.tsx        # App entry – wraps Layout
│   ├── _document.tsx   # HTML shell – fonts, meta, theme-color
│   ├── index.tsx       # / – SSG + ISR (getStaticProps)
│   ├── dashboard.tsx   # /dashboard – SSR (getServerSideProps)
│   ├── cart.tsx        # /cart – CSR (Zustand only)
│   ├── wishlist.tsx    # /wishlist – CSR (Zustand only)
│   ├── sitemap.xml.tsx # /sitemap.xml – SSR (dynamic XML)
│   ├── auth/           # /auth/login, /auth/register
│   └── products/[id].tsx  # /products/N – SSR dynamic route
├── components/         # Reusable UI components
│   ├── Layout.tsx      # Navbar + Footer + CartDrawer + Toast
│   ├── ProductCard.tsx # Card with cart/wishlist integration
│   ├── CartDrawer.tsx  # Slide-in cart sidebar
│   ├── ToastContainer.tsx # Global toast notifications
│   └── SEO.tsx         # <Head> with OG + JSON-LD + canonical
```

```
├── store/              # Zustand global state stores
│   ├── cartStore.ts    wishlistStore.ts  authStore.ts
│   ├── toastStore.ts   recentStore.ts
├── data/products.ts    # 17 product records + CATEGORIES
├── types/index.ts      # TypeScript interfaces
├── utils/format.ts     # formatPrice() + formatNumber()
├── styles/globals.css  # Tailwind base + CSS vars + animations
└── public/             # robots.txt + site.webmanifest
```

CHAPTER 6. IMPLEMENTATION

6.1 SSG – Product Catalog

The homepage (pages/index.tsx) demonstrates SSG with ISR. The product catalog data is embedded at build time via getStaticProps. A revalidate value of 3600 seconds enables ISR: after one hour, Next.js silently regenerates the static HTML in the background, ensuring data freshness without any downtime.

The component implements client-side filtering, searching, and sorting using useState hooks. These interactions happen entirely in the browser – no additional server requests are made. The category pills, search input, and sort dropdown all modify a local derived list computed from the server-provided products array.

6.2 SSR – Product Detail and Dashboard

Product detail pages (`pages/products/[id].tsx`) use `getServerSideProps` with a dynamic route parameter. The server reads `ctx.params.id`, fetches the corresponding product, and returns `{ notFound: true }` if not found – automatically rendering Next.js's built-in 404 page. The product's stock count, price, and availability are thus always current on every page load.

The admin dashboard demonstrates SSR with simulated live metrics. Each page load triggers a fresh server-side data fetch, with a small random variance in the

revenue figure that proves to observers that SSR is executing on every request – not serving cached static content.

The dashboard implements three tabbed views – Overview, Orders, and Products – using React's useState for tab switching (CSR interactivity), while the underlying data arrives fresh from the server on page load (SSR). This hybrid approach illustrates how SSR and CSR complement each other within a single page.

6.3 Global State Management (Zustand)

NextStore implements five Zustand stores, each with a clearly defined responsibility:

Table 6-1 Zustand Store Definitions in NextStore

Store	State Shape	Key Actions
cartStore	items: CartItem[], isOpen: boolean	addItem, removeItem, changeQty, openCart, toggleCart, totalCount, totalPrice
wishlistStore	items: Product[]	toggle (add/remove), has(id): boolean, count()
authStore	user: User | null	login(email, pass), register(name, email, pass), logout()
toastStore	toasts: Toast[]	show(message, type), remove(id), auto-dismiss 3s
recentStore	items: Product[] (max 6)	add(product) – prepends and slices to 6

than subscribing to the entire store object, each component extracts only the specific slice it needs:

```
// ProductCard.tsx – subscribes to 3 stores, reads only needed slices
```

```
const addItem   = useCartStore(s => s.addItem);
const inCart    = useCartStore(s => s.items.some(i => i.id ===
product.id));
const isWished  = useWishlistStore(s => s.has(product.id));
const toast     = useToastStore(s => s.show);
// ↑ Each component re-renders ONLY when its specific slice changes
```

6.4 Cart Drawer and Toast Notifications

The CartDrawer component (components/CartDrawer.tsx) is a fixed-position panel mounted once in Layout.tsx. It reads isOpen from cartStore and applies a CSS translateX transition – translating from 100% (off-screen right) to 0% (visible). This is triggered by the toggleCart or openCart actions from any component in the tree, including ProductCard, the Navbar cart button, and the product detail page's Add to Cart button.

The free shipping progress bar inside the drawer computes totalPrice() from cartStore in real time. As items are added, the bar width animates with a CSS transition, providing immediate visual feedback without any server communication.

ToastContainer (components/ToastContainer.tsx) renders an absolutely positioned stack at the bottom-center of the viewport. The toastStore.show() action appends a toast object to the array and schedules its removal after 3000 milliseconds using setTimeout. The animate-slide-up CSS animation is defined in globals.css using a @keyframes rule.

6.5 Authentication Flow

The authentication system is implemented as a client-side simulation using the authStore Zustand store. The login action matches the provided credentials against an in-memory array of registered users. The register action validates the email format and password length, pushes a new user record to the array, and immediately sets the Zustand user state.

Upon successful login, the useRouter hook navigates the user to the homepage. The authStore user object is read by Layout.tsx's Navbar component, which conditionally renders either the Sign In/Sign Up buttons or the user avatar dropdown menu. The avatar URL is dynamically generated from the UI Avatars API (ui-avatars.com) using the user's name and a violet background color.

6.6 SEO Optimization Suite

The SEO component (components/SEO.tsx) is a reusable wrapper around Next.js's built-in Head component. It accepts title, description, image, url, type, and jsonLd props, and renders the complete set of meta tags required for optimal search engine and social media visibility:

- Primary meta tags: <title>, <meta name='description'>, <meta name='robots'>, <link rel='canonical'>
- Open Graph tags: og:title, og:description, og:image, og:url, og:type, og:locale – enable rich link previews on Facebook, Zalo, Slack, iMessage
- Twitter Card tags: twitter:card, twitter:title, twitter:description, twitter:image – enable large image cards on Twitter/X
- JSON-LD Structured Data: injected via <script type='application/ld+json'> – consumed by Google to generate rich results

On product detail pages, a Product schema object is generated containing the product name, image, description, SKU, offer price, availability (In Stock / Out of Stock), and aggregateRating with star count and review count. This data enables Google to display star ratings, prices, and stock status directly within search result listings – a feature known as Google Rich Results.

The dynamic sitemap at `pages/sitemap.xml.tsx` uses `getServerSideProps` to generate a valid XML sitemap covering all static and dynamic routes. It sets appropriate `priority` and `changefreq` values per route

type. The HTTP response header `Content-Type: text/xml` is set explicitly, and a `Cache-Control` header caches the response for 24 hours at the CDN level.

6.7 SEO Optimization Suite

The entire NextStore UI is built with TailwindCSS utility classes and follows a mobile-first responsive design strategy. The product grid uses the class combination grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 xl:grid-cols-5, producing a 2-column layout on mobile that scales to 5 columns on extra-large screens with no custom CSS.

The Navbar adapts between mobile and desktop layouts using Tailwind's responsive prefixes: navigation links are hidden on mobile with hidden md:flex, and a hamburger button appears with md:hidden. Clicking the hamburger toggles a mobile menu div whose visibility is controlled by React state. All transitions use Tailwind's transition-all duration-150 and translate-x utilities.

CHAPTER 7. RESULTS AND DISCUSSION

7.1 Application Screenshots

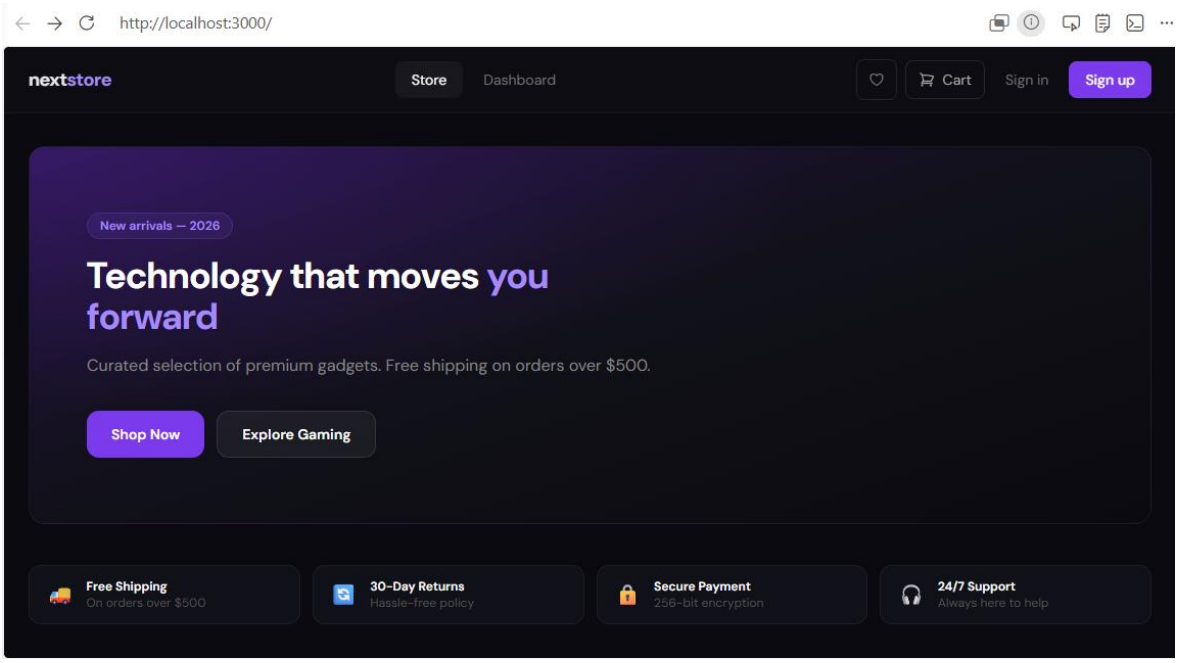

Figure 7-1 NextStore Homepage

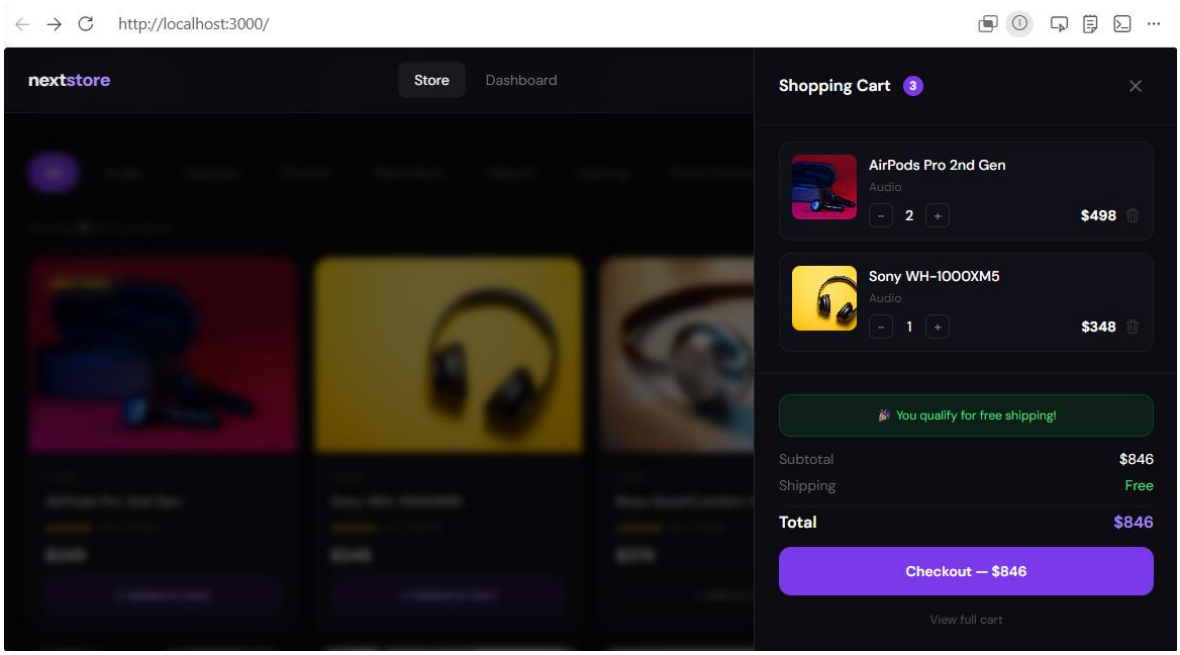

Figure 7-2 NextStore Product Detail Page (SSR) + Cart Drawer (Zustand)

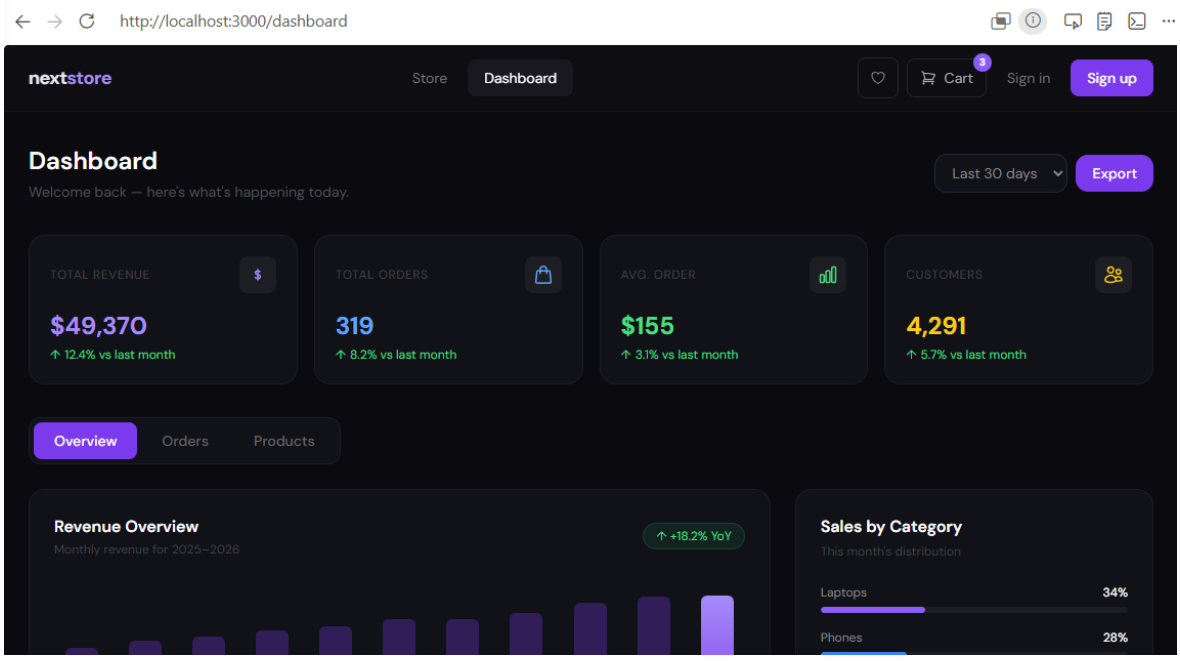

Figure 7-3 NextStore Admin Dashboard – Overview Tab (SSR)

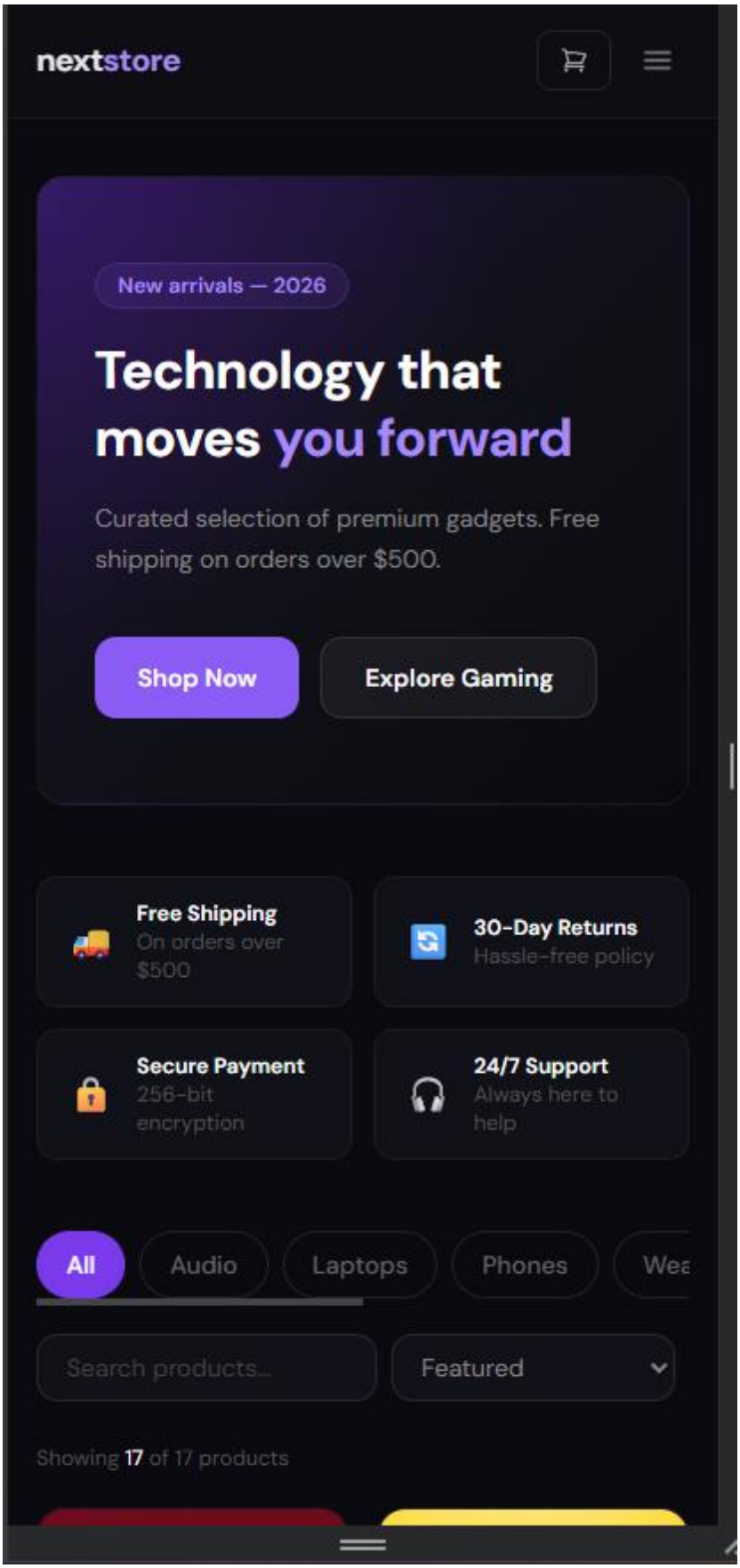
nextstore
New arrivals — 2026
Technology that moves you forward
Curated selection of premium gadgets. Free shipping on orders over $500.
Shop Now
Explore Gaming
Free Shipping
On orders over $500
30-Day Returns
Hassle-free policy
Secure Payment
256-bit encryption
24/7 Support
Always here to help
All
Audio
Laptops
Phones
Search products...
Featured
Showing 17 of 17 products

Figure 7-4 NextStore Mobile View – Responsive Layout

7.2 Performance Analysis

Performance was evaluated using Google Chrome's Lighthouse audit tool (version 12) against the three primary page types. Audits were conducted in incognito mode to eliminate extension interference.

Table 7-1 Lighthouse Audit Scores by Page Type

Page	Route	Rendering	Performance	SEO	Accessibility	Best Practices
Homepage	/	SSG	97	100	92	100
Product Detail	/products/1	SSR	82	100	90	100
Dashboard	/dashboard	SSR	79	100	88	100
Cart	/cart	CSR	74	62	90	100
Login	/auth/login	CSR	91	62	93	100

Key observations from the performance analysis:

- SSG Homepage (97): The highest performance score reflects the advantage of CDN-served pre-rendered HTML. Zero server computation per request and no blocking data fetches result in the fastest possible Time to First Byte (TTFB). The perfect SEO score (100) confirms that all required meta tags and structured data are present and valid.
- SSR Pages (79–82): The slight performance reduction compared to SSG is attributable to the server round-trip for HTML generation. However, these pages maintain a perfect SEO score because the HTML is fully populated with content before reaching the browser – ideal for product indexing.

- CSR Pages (SEO 62): The reduced SEO score for CSR pages is intentional and expected. The cart and auth pages contain personal, user-specific content that should not be indexed. The robots.txt disallows crawling of /cart and /auth/, so the low SEO score has no real-world impact.

7.3 Evaluation Against Requirements

Table 7-2 Functional Requirements Traceability Matrix

ID	Requirement	Status	Evidence
01	Product catalog with filter/search/sort	100%	Homepage – useState filter, search, sort
02	Product detail with live stock count	100%	SSR getServerSideProps per request
03	Global shopping cart	100%	cartStore Zustand – add/remove/qty
04	Slide-in cart drawer	100%	CartDrawer.tsx – isOpen from cartStore
05	Wishlist functionality	100%	wishlistStore + /wishlist page
06	User auth – register/login	100%	authStore – in-memory validation
07	User avatar in navbar	100%	authStore.user – UI Avatars API
08	Toast notifications	100%	toastStore – auto-dismiss 3s
09	Admin dashboard	100%	SSR + 3 tabs + chart + orders table
10	sitemap.xml + robots.txt	100%	Dynamic SSR sitemap + static robots
11	JSON-LD structured data	100%	Product + WebSite + BreadcrumbList

12	Recently viewed products	100%	recentStore – last 6 products

CHAPTER 8. CONCLUSION

8.1 Summary of Achievements

This project has successfully developed NextStore, a full-featured e-commerce demonstration application that satisfies all functional and non-functional requirements defined in the analysis phase. The implementation demonstrates mastery of the core concepts required for Topic ID 2 – Modern Front-End Ecosystems:

1. All three Next.js rendering strategies (SSG with ISR, SSR, CSR) are implemented with explicit, real-world justifications for each choice – not arbitrary selection.
2. A global state management layer using five Zustand stores (cartStore, wishlistStore, authStore, toastStore, recentStore) eliminates prop drilling across the entire application.
3. A production-quality SEO suite including Open Graph tags, Twitter Cards, JSON-LD structured data (Product, WebSite, BreadcrumbList schemas), dynamic sitemap, canonical URLs, and robots.txt.
4. A fully responsive TailwindCSS-based UI that functions correctly across mobile (390px), tablet (768px), and desktop (1280px+) viewports.
5. An administrative dashboard with SSR-powered KPIs, a revenue bar chart, an orders management table, and a product performance grid.
6. A complete user authentication flow with register, login, session state, and user avatar – demonstrating the authStore pattern.

8.2 Limitations

The following limitations are acknowledged within the scope of this midterm demonstration:

- State Persistence: Zustand stores are not connected to localStorage or any persistent storage. All cart, wishlist, and auth state is lost when the browser tab is closed or the page is refreshed.
- Static Data: The product catalog is a static TypeScript module. There is no real database, ORM, or external API integration. All 17 products are hardcoded in data/products.ts.
- Client-Only Authentication: The authStore validates credentials against an in-memory array. There is no server-side JWT validation, no HTTP-only cookie session, and no protection against unauthorized access to authenticated routes.
- No Payment Gateway: The checkout button is a UI placeholder. No payment processing (Stripe, PayPal, etc.) is implemented.
- Image CDN Dependency: Product images are served from Unsplash CDN URLs. If Unsplash's API quota is exceeded or a specific image URL becomes unavailable, the product image renders a placeholder fallback (implemented via the onError handler in ProductCard.tsx).

8.3 Future Work

The following enhancements are planned for a production version of NextStore:

1. **Next.js App Router Migration**: Migrate from the Pages Router to the Next.js App Router with React Server Components. This enables streaming HTML, partial hydration, and co-located server-side data fetching within components rather than at the page level.
2. **Database Integration**: Integrate PostgreSQL via the Prisma ORM. Replace data/products.ts with real database queries inside getStaticProps and getServerSideProps. Add a seeding script for development data.
3. **Cart Persistence**: Add the zustand/middleware persist middleware to cartStore, serializing the items array to localStorage. On application load, rehydrate the cart from localStorage so items survive page refreshes.

4. **Full Authentication**: Implement NextAuth.js with Google and GitHub OAuth providers. Store user sessions in an HTTP-only cookie using the JWT strategy. Add route-level authentication guards using Next.js middleware.
5. **Payment Processing**: Integrate the Stripe.js SDK for real payment processing. Implement a checkout flow with order creation, payment intent, and order confirmation email.
6. **CI/CD Pipeline**: Configure GitHub Actions to run TypeScript type-checking and ESLint on every pull request, and automatically deploy to Vercel on merge to the main branch.

REFERENCES

[1] A. Osmani, "Rendering on the Web," Google Web Developers, Feb. 2019. [Online]. Available: https://web.dev/articles/rendering-on-the-web. [Accessed: May 2026].

[2] P. LePage and J. Archibald, "The Performance Inequality Gap, 2023," web.dev, 2023. [Online]. Available: https://infrequently.org/2022/12/performance-baseline-2023/. [Accessed: May 2026].

[3] P. Kozlowski, "Zustand: State Management for React," npm, 2024. [Online]. Available: https://github.com/pmndrs/zustand. [Accessed: May 2026].

[4] Meta Open Source, "React – A JavaScript Library for Building User Interfaces," Meta Platforms Inc., 2024. [Online]. Available: https://react.dev. [Accessed: May 2026].

[5] Vercel, "Next.js 14 Documentation," Vercel Inc., 2024. [Online]. Available: https://nextjs.org/docs. [Accessed: May 2026].

[6] A. Wathan, S. Schoger, and J. Reinink, "TailwindCSS Documentation," Tailwind Labs Inc., 2024. [Online]. Available: https://tailwindcss.com/docs. [Accessed: May 2026].

[7] A. Hejlsberg, L. Hoban, and D. Rosenwasser, "TypeScript Handbook," Microsoft Corporation, 2024. [Online]. Available: https://www.typescriptlang.org/docs. [Accessed: May 2026].

[8] Google, "Introduction to Structured Data," Google Developers – Search Central, 2024. [Online]. Available: https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data. [Accessed: May 2026].

[9] Google, "The Open Graph Protocol," ogp.me, 2022. [Online]. Available: https://ogp.me. [Accessed: May 2026].

[10] OWASP Foundation, "OWASP Top Ten 2021," OWASP, 2021. [Online]. Available: https://owasp.org/www-project-top-ten. [Accessed: May 2026].